UKUDLA Machine Learning Certificate

Introduction to Machine Learning for Agri-Food Systems

Ayinde Mohammed Usman
University of the Western Cape

UKUDLA
African German Centre
for Sustainable and Resilient Food Systems and Applied Agricultural and Food Data Science

Supported by:
Federal Foreign Office

With funding from the:
Federal Ministry of Research, Technology and Space

With support from
Federal Ministry of Agriculture, Food and Regional Identity
by decision of the German Bundestag

DAAD Deutscher Akademischer Austauschdienst
German Academic Exchange Service

science, technology & innovation
Department:
Science, Technology and Innovation
REPUBLIC OF SOUTH AFRICA

NRF National Research Foundation

Table of contents

1. Introduction

2. Matrices and Vectors

3. Operations for ML

4. Special Matrices and Inverses

5. Systems of Linear Equations

6. Eigenvalues, Norms, and Vector Spaces

UKUDLA

Learning Objectives

By the end of this session, you will be able to:

- **Explain** what a matrix and vector are in the context of agri-food data
- **Identify** dimensions: rows as samples, columns as features
- **Perform** matrix operations in Python using NumPy

- **Interpret** the shape rules for matrix multiplication
- **Express** linear regression in matrix form
- **Recognise** eigenvalues and their role in PCA

No derivations required. Emphasis on **reading**, **interpreting**, and **debugging** ML notation.

UKUDLA

Why Linear Algebra, and Why Now?

Every ML model is linear algebra in disguise.

Why Linear Algebra, and Why Now?

Every ML model is linear algebra in disguise.

Prediction

A model predicts yield as
$\hat{y} = \mathbf{x}^\top \mathbf{w} + b$.

UKUDLA

Why Linear Algebra, and Why Now?

Every ML model is linear algebra in disguise.

Prediction

A model predicts yield as $\hat{y} = \mathbf{x}^\top \mathbf{w} + b$.

Training

Learning = minimising a loss over a matrix of data $\mathbf{X}$.

UKUDLA

Why Linear Algebra, and Why Now?

Every ML model is linear algebra in disguise.

Prediction

A model predicts yield as $\hat{y} = \mathbf{x}^\top \mathbf{w} + b$.

Dimensionality reduction

PCA finds directions in soil/spectral data using eigenvectors.

Training

Learning = minimising a loss over a matrix of data $\mathbf{X}$.

UKUDLA

Why Linear Algebra, and Why Now?

Every ML model is linear algebra in disguise.

Prediction

A model predicts yield as $\hat{y} = \mathbf{x}^\top \mathbf{w} + b$.

Dimensionality reduction

PCA finds directions in soil/spectral data using eigenvectors.

Training

Learning = minimising a loss over a matrix of data $\mathbf{X}$.

Computer vision

A drone image is a tensor of numbers; a CNN is a stack of matrix operations.

UKUDLA

Why Linear Algebra, and Why Now?

Every ML model is linear algebra in disguise.

Prediction

A model predicts yield as $\hat{y} = \mathbf{x}^\top \mathbf{w} + b$.

Dimensionality reduction

PCA finds directions in soil/spectral data using eigenvectors.

Training

Learning = minimising a loss over a matrix of data $\mathbf{X}$.

Computer vision

A drone image is a tensor of numbers; a CNN is a stack of matrix operations.

You will not need to derive these by hand.

You *will* need to read them, interpret them, and debug them.

UKUDLA

What We Will Cover

Foundations

- Matrices and vectors
- Indexing and dimensions
- Addition and scalar operations

Operations that matter for ML

- Transpose
- Matrix–matrix multiplication
- Matrix–vector multiplication
- Linear regression in matrix form

Special matrices and inverses

- Identity, inverse, pseudo-inverse
- Solving systems of equations

Beyond multiplication

- Eigenvalues and eigenvectors
- Norms and distances
- Vector spaces and linear transformations

Matrices — The Data Table

A **matrix** is a 2-dimensional array of numbers arranged in rows and columns.

In agri-food research, every data table is a matrix.

- Each **row** is one sample (a field, a plot, a batch).
- Each **column** is one feature (pH, rainfall, yield).

	pH	OC	N	Clay
Field 1	6.8	2.3	1.5	45
Field 2	6.2	1.8	1.2	52
Field 3	7.1	2.7	1.9	38
Field 4	5.9	1.4	0.9	61

A soil dataset: 4 samples, 4 features.
Dimensions: 4×4.

UKUDLA

Matrix Notation and Dimensions

- By convention, matrices are named with **capital letters**: $\mathbf{X}, \mathbf{A}, \mathbf{W}$.
- Dimensions are written as **rows** $\times$ **columns**.
- A general matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ has m rows and n columns.

$$\mathbf{X} = \begin{bmatrix} 6.8 & 2.3 & 1.5 & 45 \\ 6.2 & 1.8 & 1.2 & 52 \\ 7.1 & 2.7 & 1.9 & 38 \\ 5.9 & 1.4 & 0.9 & 61 \end{bmatrix} \qquad \mathbf{X} \in \mathbb{R}^{4 \times 4}$$

In ML: a feature matrix with n samples and p features is written $\mathbf{X} \in \mathbb{R}^{n \times p}$. The whole point of this course is to work with such matrices efficiently.

Matrix Indexing

Individual entries are accessed by **two indices**: x_{ij} is the value in **row** i**, column** j.

$$\mathbf{X} = \begin{bmatrix} \mathbf{6.8} & 2.3 & 1.5 & 45 \\ 6.2 & 1.8 & 1.2 & 52 \\ 7.1 & 2.7 & 1.9 & 38 \\ 5.9 & 1.4 & 0.9 & \mathbf{61} \end{bmatrix}$$

- $x_{1,1} = 6.8$ — pH of Field 1.
- $x_{4,4} = 61$ — clay content of Field 4.
- $x_{2,3} = 1.2$ — nitrogen of Field 2.

Accessing $x_{i,j}$ is the same as `X[i-1, j-1]` in Python (0-indexed). Getting this right is a common source of off-by-one bugs in ML code.



UKUDLA

Vectors — A Single Sample or a Single Feature

A **vector** is a matrix with only one column (by convention) — an $n \times 1$ matrix.

Named with a **lowercase bold letter**: $\mathbf{x}, \mathbf{w}, \mathbf{y}$.

Two views of a vector in agri-food ML:

- **A row of X:** all features of one field.
- **A column of X:** one feature across all fields.

A single sample (Field 3):

$$\mathbf{x}_3 = \begin{bmatrix} 7.1 \\ 2.7 \\ 1.9 \\ 38 \end{bmatrix}$$

Dimensions: 4×1.

UKUDLA

Vector Indexing

A vector is indexed by a **single number**: x_i is the i-th element.

$$\mathbf{y} = \begin{bmatrix} 4.5 \\ 3.8 \\ 5.1 \\ 4.2 \end{bmatrix} \quad \text{(maize yield in t/ha for four fields)}$$

- $y_1 = 4.5$ — yield of Field 1.
- $y_3 = 5.1$ — yield of Field 3.
- y_5 **does not exist** — the vector has only 4 elements.

In ML, **y** is called the **target** or **label** vector. It is what the model tries to predict.

Matrix Addition and Subtraction

Two matrices can be added or subtracted **only if their dimensions match**. Then, corresponding entries are added or subtracted.

$$\begin{bmatrix} 6.8 & 2.3 \\ 6.2 & 1.8 \end{bmatrix} + \begin{bmatrix} 0.2 & 0.4 \\ 0.1 & 0.3 \end{bmatrix} = \begin{bmatrix} 7.0 & 2.7 \\ 6.3 & 2.1 \end{bmatrix}$$

Agri-food example: A sensor reading drift correction. Suppose $\mathbf{X}$ holds raw soil moisture readings from a sensor array, and $\mathbf{D}$ holds the known drift offsets (one per sensor per day). The corrected readings are $\mathbf{X} - \mathbf{D}$.

Dimension mismatch = error

$\begin{bmatrix} 6.8 & 2.3 \\ 6.2 & 1.8 \end{bmatrix} + \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$ **is undefined.** Python raises a `ValueError`.

UKUDLA

Scalar Multiplication and Division

A **scalar** is a single number. Multiplying a matrix by a scalar multiplies every entry.

$$2 \times \begin{bmatrix} 6.8 & 2.3 \\ 6.2 & 1.8 \end{bmatrix} = \begin{bmatrix} 13.6 & 4.6 \\ 12.4 & 3.6 \end{bmatrix} \qquad \begin{bmatrix} 6.8 & 2.3 \\ 6.2 & 1.8 \end{bmatrix} / 2 = \begin{bmatrix} 3.4 & 1.15 \\ 3.1 & 0.9 \end{bmatrix}$$

Agri-food examples:

- Convert yields from t/ha to kg/ha: multiply $\mathbf{y}$ by 1000.
- Normalise rainfall to mm/day: divide the $\mathbf{X}$ column by the number of days.
- Convert temperatures from °C to °F: $\mathbf{X}_F = 1.8\mathbf{X}_C + 32$.

ML connection: *Feature scaling* (standardisation, min–max) is precisely scalar multiplication on each column of $\mathbf{X}$.

UKUDLA

Matrix Transpose

The **transpose** of a matrix $\mathbf{A}$, written $\mathbf{A}^\top$, swaps rows and columns.

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \Longrightarrow \quad \mathbf{A}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Agri-food example:

- $\mathbf{X} \in \mathbb{R}^{n \times p}$: n fields, p features.
- $\mathbf{X}^\top \in \mathbb{R}^{p \times n}$: p features, n fields.

Where you will see it:

- Computing sample covariance: $\mathbf{X}^\top \mathbf{X}$.
- The normal equations for linear regression: $\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.
- Dot product of two vectors: $\mathbf{x}^\top \mathbf{y}$.

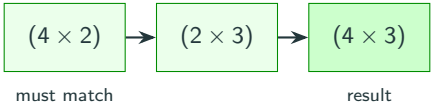
UKUDLA

Matrix–Matrix Multiplication

To multiply **AB**:

- The **number of columns of A** must equal the **number of rows of B**.
- The result has shape (rows of **A**) $\times$ (columns of **B**).

$$\underbrace{(4 \times 2)}_{\mathbf{A}} \; \underbrace{(2 \times 3)}_{\mathbf{B}} = \underbrace{(4 \times 3)}_{\mathbf{AB}}$$



Matrix–Matrix Multiplication — Worked Example

Compute **AB** where

$$\mathbf{A} = \begin{bmatrix} 1 & 4 \\ 4 & 1 \\ 2 & 3 \\ 3 & 0 \end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix} 3 & 5 & 1 \\ 2 & 6 & 0 \end{bmatrix}$$

Shape check: $(4 \times 2)(2 \times 3) \to (4 \times 3)$.

Row 1 of A, all columns of B:

$$[1,4] \cdot \begin{bmatrix} 3 \\ 2 \end{bmatrix} = 1(3) + 4(2) = 11, \quad [1,4] \cdot \begin{bmatrix} 5 \\ 6 \end{bmatrix} = 1(5) + 4(6) = 29, \quad [1,4] \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1(1) + 4(0) = 1.$$

Repeating for all rows:

$$\mathbf{AB} = \begin{bmatrix} 11 & 29 & 1 \\ 14 & 26 & 4 \\ 12 & 28 & 2 \\ 9 & 15 & 3 \end{bmatrix}$$

UKUDLA

Matrix–Vector Multiplication — Prediction in ML

Multiplying a matrix by a vector is a special case where the second operand is a single-column matrix.

$$\underbrace{(2 \times 4)}_{\mathbf{X}} \; \underbrace{(4 \times 1)}_{\mathbf{w}} = \underbrace{(2 \times 1)}_{\mathbf{Xw}}$$

Worked example:

$$\begin{bmatrix} 3 & 5 & 1 & 4 \\ 2 & 6 & 0 & 7 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 3(1) + 5(4) + 1(2) + 4(3) \\ 2(1) + 6(4) + 0(2) + 7(3) \end{bmatrix} = \begin{bmatrix} 37 \\ 47 \end{bmatrix}$$

Why this matters: this is *exactly* what a linear model computes. Every row of **X** is one field; **w** is the vector of feature weights; the output is the predicted value for each field.

UKUDLA

Matrix Multiplication Is Not Commutative

In general, $\mathbf{AB} \neq \mathbf{BA}$. In fact, one product may be defined while the other is not.

$$\mathbf{A} = \begin{bmatrix} 1 & 4 \\ 4 & 1 \\ 2 & 3 \\ 3 & 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 3 & 5 & 1 \\ 2 & 6 & 0 \end{bmatrix}$$

- **AB**: $(4 \times 2)(2 \times 3) \to (4 \times 3)$ **Defined.**
- **BA**: $(2 \times 3)(4 \times 2)$ **Undefined** $(3 \neq 4)$.

Agri-food takeaway: When composing transformations — e.g., first normalise, then project — order matters. Always check the shape of what you write in Python.

UKUDLA

The Identity Matrix

The **identity matrix I** is a square matrix with 1s on the diagonal and 0s elsewhere.

$$\mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Property: $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$ for any compatible $\mathbf{A}$. It is the matrix equivalent of the number 1.

ML connection: regularisation adds a small multiple of $\mathbf{I}$ to a matrix to make it invertible and stabilise training — e.g., Ridge regression solves $(\mathbf{X}^\top\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}$.

The Matrix Inverse

For a square matrix $\mathbf{A}$, its **inverse** $\mathbf{A}^{-1}$ satisfies $\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$.

Scalar analogy: $4 \times \frac{1}{4} = 1$. For matrices, $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$.

Worked example:

$$\mathbf{A} = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix} \quad \Longrightarrow \quad \mathbf{A}^{-1} = \frac{1}{4(6) - 7(2)} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \frac{1}{10} \begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix}.$$

Not every matrix has an inverse:

- Non-square matrices have no inverse.
- Singular (collinear) matrices have no inverse — e.g., if two columns of $\mathbf{X}$ contain the same information.

Agri-food warning

Collinearity is common in agric datasets (e.g., rainfall and humidity). The matrix $\mathbf{X}^\top\mathbf{X}$ becomes non-invertible, and linear regression breaks. Solutions: drop a feature, use Ridge, or use the



ie.

The Pseudo-Inverse

When a matrix has no inverse (non-square or singular), we use the **Moore–Penrose pseudo-inverse** $\mathbf{A}^+$.

Key property: $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}$. It behaves like an inverse in the least-squares sense.

For a tall matrix $\mathbf{X}$ ($n > p$):

$$\mathbf{X}^+ = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top.$$

ML connection: the least-squares solution to linear regression is

$$\hat{\boldsymbol{\theta}} = \mathbf{X}^+\mathbf{y} = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}.$$

This is what `np.linalg.lstsq` and `sklearn.LinearRegression` compute internally.

Systems of Linear Equations

Many ML problems reduce to solving $\mathbf{Ax} = \mathbf{b}$.

Example: On three plots, we apply different amounts of nitrogen x (kg/ha) and observe yield y (t/ha). Suppose the underlying relation is $y = a + bx$. We have three observations:

$$a + b \cdot 60 = 3.2, \quad a + b \cdot 90 = 4.1, \quad a + b \cdot 120 = 4.9.$$

In matrix form:

$$\underbrace{\begin{bmatrix} 1 & 60 \\ 1 & 90 \\ 1 & 120 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 3.2 \\ 4.1 \\ 4.9 \end{bmatrix}}_{\mathbf{b}}.$$

Note: the system is *overdetermined* (3 equations, 2 unknowns). There is no exact solution, but the least-squares solution is $\mathbf{x} = \mathbf{A}^{+}\mathbf{b}$.

UKUDLA

Solving Systems in Practice

Square, invertible

$\mathbf{Ax} = \mathbf{b} \Rightarrow \mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.
Rarely done this way in practice (numerically unstable).

Numerical solutions

Gaussian elimination, LU, QR, SVD. Python: `np.linalg.solve`, `np.linalg.lstsq`.

Agri-food use: calibrating sensor readings against reference measurements. Sensors read s, reference gives r; we fit $r = a + bs$. Matrix form solves this in one line.

UKUDLA

Eigenvalues and Eigenvectors

For a square matrix $\mathbf{A}$, an **eigenvector** $\mathbf{v}$ and **eigenvalue** λ satisfy:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

Interpretation: $\mathbf{A}$ stretches $\mathbf{v}$ by λ without rotating it.

Worked example:

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \quad \Longrightarrow \quad \lambda_1 = 2,\ \lambda_2 = 3, \quad \mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix},\ \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Why this matters in ML: Principal Component Analysis (PCA) finds the eigenvectors of the covariance matrix of $\mathbf{X}$. The eigenvalues tell you *how much variance* each component explains. In agri-food, PCA is used to reduce multi-spectral satellite bands or soil-property measurements to a few interpretable axes.

Preview: PCA on Soil Data

You measure 10 soil properties for 500 fields. They are highly correlated. PCA finds *new axes* (components) that capture most of the variation with fewer numbers.

Component	Eigenvalue	% Variance	Cumulative %
PC1	5.4	54%	54%
PC2	2.1	21%	75%
PC3	1.2	12%	87%
PC4	0.6	6%	93%
...	...	...	...

We can keep only 3 components (87% of variance) instead of 10 variables. Used in clustering fields into soil-management zones.

Vector Spaces and Basis

A **vector space** is a set of vectors closed under addition and scalar multiplication.

A **basis** is a minimal set of vectors that spans the space — i.e., any vector can be written as a linear combination of the basis vectors.

ML connection:

- Feature vectors live in $\mathbb{R}^p$.
- Models find a useful *basis* for prediction.
- Neural network layers compute new bases through learned weights.

Agri-food example: In a spectral dataset, raw bands (B2, B3, B4, ...) form one basis. Vegetation indices (NDVI, EVI, SAVI) form another, more informative basis for crop health prediction.

Norms and Distance Metrics

A **norm** measures the length of a vector. The most common is the Euclidean norm:

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_p^2}.$$

Worked example:

$$\mathbf{x} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} \quad \Longrightarrow \quad \|\mathbf{x}\|_2 = \sqrt{9 + 16} = 5.$$

Agri-food examples:

- **Similarity of two soil samples:** $\|\mathbf{x}_i - \mathbf{x}_j\|_2$.
- **Anomaly detection:** fields whose feature vector is far from the average are candidates for review.
- **Loss functions:** MSE is the squared Euclidean norm of residuals.
- **Regularisation:** L1 (Lasso) and L2 (Ridge) use ℓ_1 and ℓ_2 norms.

Linear Transformations

A matrix is a **transformation**: it stretches, rotates, or projects vectors.

Examples in agri-food ML:

- **Rotation:** aligning drone imagery with a map.
- **Projection:** PCA projects high-dimensional soil data onto a 2D plane.
- **Scaling:** standardising features across fields.
- **Convolution:** a CNN layer applies many small matrix transformations to an image.

Key insight: A deep neural network is a *composition* of linear transformations interleaved with non-linearities. The linear part is all matrices.

Key Takeaways

1. Every agri-food dataset is a matrix: rows are samples, columns are features.

Key Takeaways

1. Every agri-food dataset is a matrix: rows are samples, columns are features.

2. Linear regression, PCA, and neural networks are all built from matrix operations.

Key Takeaways

1. Every agri-food dataset is a matrix: rows are samples, columns are features.

2. Linear regression, PCA, and neural networks are all built from matrix operations.

3. Shape rules protect you: $(n \times p)(p \times 1) = (n \times 1)$. Always check shapes.

Key Takeaways

1. Every agri-food dataset is a matrix: rows are samples, columns are features.

2. Linear regression, PCA, and neural networks are all built from matrix operations.

3. Shape rules protect you: $(n \times p)(p \times 1) = (n \times 1)$. Always check shapes.

4. Eigenvalues and norms are the mathematical foundations of PCA, anomaly detection, and loss functions.

Supported by

South African government and the National Research Foundation

science, technology & innovation
Department:
Science, Technology and Innovation
REPUBLIC OF SOUTH AFRICA

NRF National Research Foundation

German government and the German Academic Exchange Service

Supported by:
Federal Foreign Office

With funding from the:
Federal Ministry of Research, Technology and Space

Supported by:
Federal Ministry of Agriculture, Food and Regional Identity
by decision of the German Bundestag

Project Manager:
Federal Office for Agriculture and Food

DAAD
Deutscher Akademischer Austauschdienst
German Academic Exchange Service

UKUDLA

Reference

- **Brunton, S. L., & Kutz, J. N.** (2019). *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press.
  - Chapter 1 (linear algebra) is directly relevant.
  - Accompanying code: `databookuw.com`.
- **Deisenroth, M. P., Faisal, A. A., & Ong, C. S.** (2020). *Mathematics for Machine Learning*. Cambridge University Press.
  - Free PDF: `mml-book.github.io`.
  - Chapters 2–4 cover everything in this session.
- **Strang, G.** (2016). *Introduction to Linear Algebra*, 5th ed. Wellesley–Cambridge Press.
  - The classic; excellent intuition for vectors and matrices.
- **Goodfellow, I., Bengio, Y., & Courville, A.** (2016). *Deep Learning*. MIT Press.
  - Chapter 2 is a concise linear algebra primer for ML.
- **Hastie, T., Tibshirani, R., & Friedman, J.** (2009). *The Elements of Statistical Learning*, 2nd ed. Springer.